COMPUTER SCIENCE + COMPUTER ENGINEERING

# Gate A: Training Yard

First-game specification and review standard

The exact world rules, failure behavior, persistence and replay contracts, required tests, integration milestones and handoff evidence.

**One small game · Explicit rules · Independent implementation**

Package 02 · Expanded teaching edition · September 28, 2026
Volume 01 of the full undergraduate-to-graduate program
Self-study material. Completion requires implementation, testing and review.

# Contents

Use the linked contents or PDF bookmarks to navigate.

**Gate A: Training Yard** ..... 3
- Purpose and boundary ..... 3
- World specification ..... 3
- Transition table ..... 4
- Invariants ..... 4
- Architecture and files ..... 5
- Save/load contract ..... 5
- Replay contract ..... 5
- Required tests ..... 6
- Integration milestones ..... 6
- Submission contents ..... 7
- Review rubric ..... 7
- Oral review prompts ..... 8
- Reference commands ..... 8

# Gate A: Training Yard

## Purpose and boundary

Build a complete small turn-based terminal game and explain it. The supplied reference is a comparison artifact, not a student submission. The student must implement independently or make substantial independently understood changes, write their own tests and pass the review. Running a supplied demonstration alone does not complete the project.

This specification extends the simple x/y/won helpers from Lessons 05.1-05.3. Those helpers intentionally omit energy, file I/O and interface commands. Do not silently alter their existing lab contracts to add the entire capstone at once.

## World specification

| Item | Required baseline |
|---|---|
| Map | 10 columns by 10 rows; integer coordinates 0..9 |
| Convention | Origin upper left; x grows right and y grows down |
| Start | (0,0), energy 25, accepted-action turns 0, status playing |
| Exit | (9,9) |
| Obstacles | (1,1), (3,2), (4,4), (7,6), (8,8) |
| Controls | w up; a left; s down; d right; wait; r restart; q quit |
| Presentation | P player, E exit, # obstacle, . open; show energy, turns and status |
| Rule identity | training-yard-v1 for the unchanged baseline |

A meaningful extension is required for student ownership, but the baseline must work first. Keep the extension in a separate rule version or explicitly labeled feature. Examples include configurable controls with validated bindings, a second deterministic map, or a rule-neutral replay inspector. Record new tests and changed assumptions.

# Transition table

| Situation | Required result |
| --- | --- |
| Known move to an open in-bounds cell | Move once; spend one energy; add one turn |
| Wait while playing | Keep position; spend one energy; add one turn |
| Edge or obstacle blocks movement | Preserve all simulation fields; explain rejection |
| Unknown command | Preserve simulation fields; explain supported input |
| Goal entered | Set status won; goal arrival wins even at zero energy |
| Energy becomes zero away from goal | Set status lost |
| Movement/wait after won, lost or quit | No simulation change |
| r in any status | Create a fresh starting state |
| q while playing | Set quit without spending energy; terminal exits |
| q after a terminal result | Preserve result state; terminal still exits |
| help, save, load, export | Interface operations; not charged as movement turns |

A move must be validated as a whole before changing position or resources. Diagnostic messages are output, not persisted simulation fields. Do not infer victory by inspecting the rendered characters.

# Invariants

The player is in bounds and not in an obstacle. Energy and turns are integers, not Boolean flags. Within a round, energy = 25 - turns, with 0 <= turns <= 25. Won implies goal position and goal position implies won. Lost requires zero energy away from the goal. Playing and quit require positive energy. A new round resets all round-local fields.

These checks establish schema consistency, not complete historical reachability of every possible imported position. The educational reference validates these declared invariants; it does not prove that every accepted save can arise from every particular command history.

# Architecture and files

Separate the state/update model, rendering, terminal interface and persistence. The update accepts explicit state and command and returns a result without prompting, printing or reading the clock. Rendering observes state. Input parsing and filesystem operations belong at the edges. Use small functions and a clear data representation; no particular class hierarchy is required.

The supplied reference lives in `capstone/reference/yard/`. The student scaffold lives in `capstone/student/`. Do not import the reference into student code as a way of passing. A differently organized student project is acceptable when it documents an equivalent executable entry point and tests.

# Save/load contract

Use a versioned JSON envelope containing version, rules and state. The state fields are x, y, energy, turns and status. Reject unsupported versions/rules, missing or extra fields, wrong types, invalid ranges and inconsistent states. The reference rejects duplicate keys, non-finite JSON constants, non-UTF-8 files and files larger than 65,536 bytes.

Parse and validate a candidate before replacing live state. Failed load leaves the previous state usable. A save must not replace an existing valid save with partially written content after an ordinary failed-write path. The reference writes a same-directory temporary file and replaces on success; this is not a universal power-loss durability guarantee, concurrent-writer database or safe untrusted-directory sandbox. Keep experiments in your own disposable workspace. [S11, S15]

An equivalent student format is acceptable when it is explicitly specified and equally tested. Do not claim binary compatibility with the supplied reference format unless demonstrated.

# Replay contract

Record an initial state, rule/map identity, format version and ordered commands. A replay using the same defined conditions must reproduce the expected final state. Preserve a twenty-command fixture. The supplied fixture uses nine d, nine s, then a and ? after victory; those final commands leave the won state unchanged.

The reference supports at most 1,000 commands per replay and limits each command to 256 characters. A successful interactive load begins a new segment from the loaded state. A full in-memory replay buffer begins a new segment from the current state. Export includes that segment's actual initial state. Save, load, help and export themselves are interface operations rather than recorded movement actions.

Changing the rules or map requires an explicit compatibility policy. Do not silently reinterpret old recordings and then call them deterministic. A digest is optional evidence of defined state bytes, not a correctness proof. [S18]

# Required tests

Write at least twelve learner-owned tests. Cover the categories below; combine related parameterized cases only when the test remains understandable. The supplied reference tests are examples, not a replacement for this requirement.

1. Starting state and fresh independent restart.
2. One valid move causes one update.
3. All four edges reject movement.
4. All five obstacles reject movement.
5. Rejection leaves position, energy, turns and status unchanged.
6. Repeated rendering has no state effects.
7. Unknown and normalized commands follow the policy.
8. Known route wins with energy 7 after 18 actions.
9. Waiting 25 times loses away from the goal.
10. Seven waits plus the known route wins on zero energy.
11. Terminal movement is a no-op, and restart works after win/loss.
12. Valid save round trip and failed-load preservation.
13. Malformed, incompatible and inconsistent saves are rejected.
14. A failed-write experiment preserves the previous save in its stated conditions.
15. Repeated replay agrees with an independently predicted state.
16. Clean launch, quit, missing-file handling and test invocation work.

At least one test must detect a seeded defect in a disposable copy. Restore the correct code afterward. Include at least five learner-chosen additional boundary/failure inputs, not just copied demonstrations.

# Integration milestones

**M1 - Pure world:** establish coordinates, commands, candidate update and read-only render. Check with small deterministic inputs.

**M2 - Complete rules:** add obstacles, energy, goal, loss, terminal no-op and restart. Test the final-energy tie and partial-mutation risks.

**M3 - Persistence and replay:** specify schema and rule version, validate candidates, test failures and preserve a known twenty-command fixture.

**M4 - Tested release:** write learner-owned tests, run a mutation experiment, package a clean source release and verify the README.

**M5 - Independent review:** observe a tester without coaching, record three concrete observations, fix at least one usability issue and defend a live change.

These milestones overlap the Module 05 project; they are not five additional unrelated capstones.

# Submission contents

Supply source, README, explicit rules and invariants, learner tests with commands/results, save/replay schema, a replay fixture and expected state, raw benchmark samples with boundaries, a journal explaining two bugs, provenance/assistance notes, tester observations, release checklist and a short technical explanation. Do not include private files, credentials, virtual environments or unrelated caches.

The performance report must describe the work timed, environment and variability. It does not require an invented speed threshold. A terminal Python microbenchmark is not evidence that a graphics engine will meet a frame-rate target.

# Review rubric

| Category | Points |
|---|---|
| Correctness and required behavior | 30 |
| Tests and verification | 25 |
| Reasoning and design tradeoffs | 15 |
| Quantitative analysis | 10 |
| Reproducibility | 10 |
| Communication and provenance | 10 |

Suggested pass: 80/100 plus all essential rules and failure checks. A polished interface cannot compensate for silent state corruption, a nonworking restart or false claims about tests. Mark independent review as pending when it has not actually occurred.

# Oral review prompts

Explain the complete path from a typed command to the next displayed state. Show why a blocked move cannot partially consume energy. Identify the schema checks that reject a contradictory save. Reproduce the recorded session without interactive input. Predict the tests affected by a new obstacle or changed energy cap, then implement a small variation. Explain one failed attempt and how the evidence located the defect.

# Reference commands

Run from the package root, using the verified launcher for your operating system:

```
python3 capstone/reference/play.py --demo
python3 capstone/reference/play.py
python3 capstone/reference/play.py --replay data/winning_replay.json
python3 tools/check_game.py
python3 tools/benchmark_game.py --updates 10000 --trials 7
```

The interactive reference stores explicit saves and exported recordings in `workspace/` beneath the working directory. It does not autosave on quit. The student must document their own equivalent behavior. These commands validate the reference only; run the student's own tests for the student project.